

Chapter 1 — Foundations of Continuous Integration, Continuous Delivery, and Jenkins

1.1 Introduction

Modern software systems are expected to change quickly without becoming unstable. Teams are asked to release new features frequently, fix defects rapidly, and maintain quality across increasingly complex applications. In such environments, manual build and deployment practices are not enough.

This is where **Continuous Integration (CI)** and **Continuous Delivery (CD)** become essential. They introduce automation into the software delivery process so that code changes are built, tested, packaged, and prepared for release in a predictable way.

Jenkins is one of the most widely used automation servers for implementing CI/CD. It acts as the orchestration layer that connects source control, build tools, test frameworks, container tools, and deployment systems into a single delivery flow.

1.2 Why CI/CD Is Needed?

Traditional software delivery often depends on manual work at several stages:

- code is merged manually,
- builds are triggered manually,
- tests are run inconsistently,
- deployment steps are repeated by hand,
- and environment mismatches cause unexpected failures.

As projects grow, this approach becomes error-prone and slow. A change that worked on a developer's machine may fail during integration, testing, or deployment because the process is not standardised.

CI/CD solves this problem by turning delivery into an automated workflow. Every code change passes through the same pipeline, which improves consistency and reduces risk.

1.3 Continuous Integration

1.3.1 Definition

Continuous Integration is the practice of merging code changes frequently into a shared repository and verifying each integration through an automated build and test process.

The main idea is simple: the earlier a defect is detected, the cheaper and easier it is to fix.

1.3.2 CI Workflow

A typical CI flow looks like this:

```
Developer commits code
↓
Source control is updated
↓
Jenkins detects the change
↓
Build starts automatically
↓
Tests execute
↓
Result is reported
```

1.3.3 Benefits of CI

Continuous Integration provides several important advantages:

- early detection of build failures,
- faster feedback to developers,
- reduced merge conflicts,
- improved code confidence,
- and a repeatable validation process.

Instead of waiting until the end of a development cycle, issues are discovered continuously as code is integrated.

1.3.4 CI for MERN application

We will apply the CI process to a MERN-based application with supporting automation tools. Jenkins will retrieve the source code, run build steps, execute automated tests, and prepare the project for later delivery stages.

1.4 Continuous Delivery

1.4.1 Definition

Continuous Delivery extends Continuous Integration by ensuring that the software remains in a deployable state at all times.

In a Continuous Delivery process, code that passes the build and test stages is packaged and prepared for release. The final production deployment may still require a manual approval step.

1.4.2 Delivery Pipeline

A delivery pipeline is the structured path from source code to a release-ready software artifact.

A typical pipeline includes:

```
Source -> Build -> Test -> Package -> Release -> Deploy
```

Each stage acts as a quality checkpoint. If one stage fails, the pipeline stops and the issue is visible immediately.

1.4.3 Continuous Delivery vs Continuous Deployment

These terms are often confused, but they are not the same.

Aspect	Continuous Delivery	Continuous Deployment
Build and test automation	Yes	Yes
Artifact preparation	Yes	Yes
Production release	Manual decision	Automatic
Human approval	Present	Not required
Risk profile	Lower	Higher

For most enterprise systems, Continuous Delivery is the preferred model because it gives teams confidence while still keeping a human release gate.

1.5 What Is a Delivery Pipeline?

A delivery pipeline is the automated sequence that transforms source code into a deployable software release.

It is designed to provide:

- repeatability,
- traceability,
- visibility,
- quality control,
- and release confidence.

Instead of treating build and deployment as isolated tasks, the pipeline makes them part of one managed workflow.

1.6 Stages of a Delivery Pipeline

1.6.1 Source Stage

The source stage pulls the latest code from version control, usually Git.

Typical actions include:

```
git clone
git pull
```

This stage ensures the pipeline always works on the required revision of the codebase.

1.6.2 Build Stage

The build stage compiles source code, resolves dependencies, and prepares the application for testing or packaging.

Examples:

- Java projects use Maven
- Node.js projects use npm
- Frontend builds may produce distributable static assets
- Docker builds produce container images

Example Maven command:

```
mvn clean package
```

Example Node.js build command:

```
npm install
npm run build
```

1.6.3 Test Stage

The test stage validates the application automatically.

This may include:

- unit tests,
- integration tests,
- API tests,
- UI tests,
- and browser automation tests using Selenium.

Testing is the critical quality gate in a pipeline. A failed test stage usually means the code must be fixed before moving further.

1.6.4 Package Stage

The package stage produces a reusable artifact.

Examples include:

- `.jar`
- `.war`
- Docker image
- compressed release bundle

Example:

```
docker build -t myapp:1.0 .
```

1.6.5 Release Stage

The release stage publishes the validated build artifact to a repository or registry.

Examples:

- Docker Hub
- Nexus Repository
- Artifactory
- internal package store

This makes the artifact available for deployment or future reuse.

1.6.6 Deploy Stage

The deploy stage moves the artifact into an environment such as development, staging, or production.

Deployment may be performed through:

- Docker container launch,
 - Kubernetes rollout,
 - shell scripts,
 - Ansible,
 - or another deployment mechanism.
-

1.7 Introduction to Jenkins

1.7.1 What Jenkins Is?

Jenkins is an open-source automation server used to implement CI/CD workflows.

It is designed to automate tasks such as:

- building applications,
- running tests,

- packaging releases,
- deploying software,
- and integrating with many development tools.

Jenkins is especially popular because it is flexible, extensible, and supported by a very large plugin ecosystem.

1.7.2 Why Jenkins Is Widely Used?

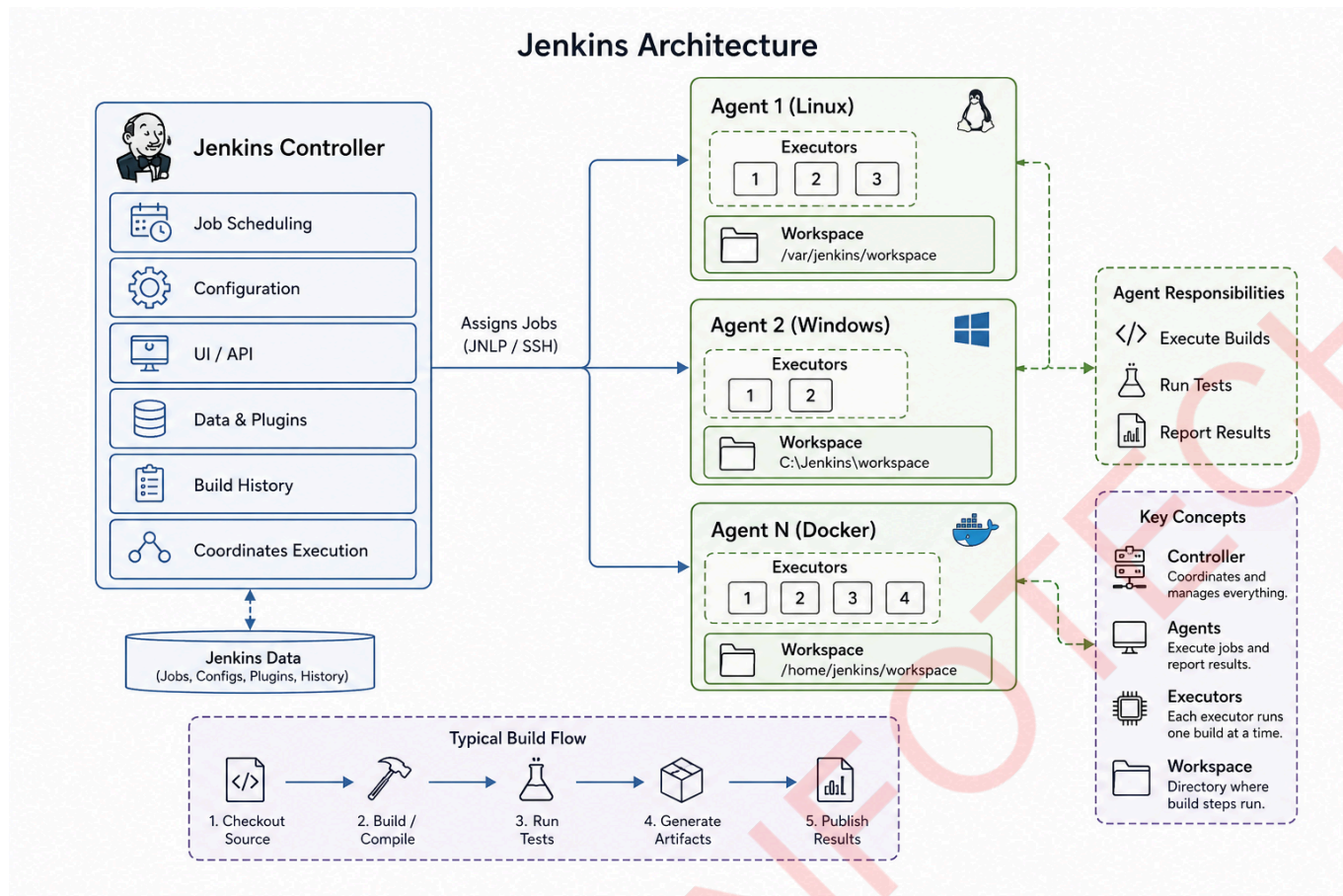
Jenkins became an industry standard for automation because it offers:

Feature	Value
Open-source platform	No licensing cost
Large plugin ecosystem	Integrates with many tools
Pipeline support	Enables pipeline-as-code
Distributed execution	Supports multiple agents
Cross-platform	Works across operating systems
Mature community	Stable ecosystem and documentation

For practical CI/CD learning, Jenkins remains one of the best tools to understand the full delivery lifecycle.

1.8 Jenkins Architecture

Jenkins follows a distributed architecture.



1.8.1 Controller

The Jenkins controller is the central coordination point.

It is responsible for:

- serving the Jenkins UI,
- scheduling jobs,
- storing configuration,
- managing plugins,
- tracking build history,
- and dispatching work to agents.

The controller should remain lightweight and focused on orchestration.

1.8.2 Agent

An agent is a remote execution node that runs jobs assigned by the controller.

Agents are useful because they:

- offload build work from the controller,
- support different operating systems,
- allow parallel builds,
- and provide specialised execution environments.

In older Jenkins terminology, agents were often called **slave nodes**. Jenkins now uses the term **agent** more commonly.

1.8.3 Executors

An executor is a slot that can run one build at a time.

If a node has:

- 1 executor, it can run 1 job concurrently,
- 2 executors, it can run 2 jobs concurrently,
- and so on.

Executors determine the practical build capacity of a node.

1.8.4 Workspace

A workspace is the directory where Jenkins checks out the source code and performs build actions.

Typical workspace operations include:

- source checkout,
 - compilation,
 - test execution,
 - artifact generation,
 - and report collection.
-

1.9 Jenkins Plugins

The Jenkins core installation is intentionally minimal. Most real-world functionality comes from plugins.

Plugins are used for:

- Git integration,
- pipeline support,
- Docker integration,
- Maven support,
- test reporting,
- notifications,
- credentials,
- and access control.

Without plugins, Jenkins would remain a basic automation shell. With plugins, it becomes a full delivery platform.

1.10 Freestyle Jobs and Pipeline Jobs

1.10.1 Freestyle Jobs

Freestyle jobs are configured through the Jenkins UI.

They are useful for simple or legacy tasks, but they have limitations:

- they are harder to version control,
- they are not easily reproducible,
- they do not scale well for complex delivery logic,
- they do not fit modern pipeline-as-code practice.

1.10.2 Pipeline Jobs

Pipeline jobs define the delivery workflow in code.

The pipeline definition is usually stored in a file called `Jenkinsfile`.

Advantages of pipeline jobs:

- stored in version control,
- easier to review,
- reusable across branches and environments,
- suitable for complex workflows.

Pipeline jobs are the modern standard for CI/CD.

1.11 Declarative and Scripted Pipeline

Jenkins supports two pipeline styles:

- Declarative Pipeline
- Scripted Pipeline

Declarative Pipeline is preferred for most business and academic workflows because it is more structured and easier to read. It uses a clear syntax with `pipeline`, `agent`, `stages`, and `steps`.

Example:

```
pipeline {
  agent any

  stages {
    stage('Build') {
      steps {
        sh 'mvn clean package'
      }
    }

    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
  }
}
```

```
}  
}  
}
```

Scripted Pipeline gives more programming flexibility, but it is less structured and usually harder to maintain.

1.12 Installing Jenkins on Ubuntu 22.04

Possible Jenkins installation options:

1. Docker image
2. Native installation (on Windows, Linux or Mac)

We will install on Ubuntu 22.04 - Single node installation.;

- Jenkins 2.555.2 (Latest)
- Java 21

A stable Jenkins installation starts with installing the correct Java runtime and then adding Jenkins from the official repository.

1.12.1 Install Java

Jenkins requires a compatible Java version.

```
sudo apt update  
sudo apt install fontconfig openjdk-21-jre -y  
java -version
```

1.12.2 Add the Jenkins Repository

```
sudo mkdir -p /etc/apt/keyrings  
  
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \  
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key  
  
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] \  
https://pkg.jenkins.io/debian-stable binary/" | \  
sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
```

1.12.3 Install Jenkins

```
sudo apt update
sudo apt install jenkins -y
```

1.12.4 Start and Verify the Service

```
sudo systemctl status jenkins
```

If the service is running correctly, the status should show it as "active".

1.12.5 Open Jenkins in the Browser

Jenkins typically runs on port 8080.

```
http://<server-ip>:8080
```

1.12.6 Unlock Jenkins

The initial admin password can be read with:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

This password is used only for the first-time setup wizard.

1.12.7 Install Suggested Plugins

During first launch, Jenkins prompts for plugin installation. Choosing the suggested plugin set is the fastest way to get a usable starting environment.

1.12.8 Create the First Admin User

After plugins are installed, the setup wizard prompts for:

- username,
- password,
- full name,
- email address.

This admin account becomes the first user of the system.

1.13 Jenkins Home Directory

The Jenkins home directory stores configuration, job data, plugins, and build history.

Default location: `/var/lib/jenkins`

Common contents include:

Directory	Purpose
jobs/	Job configurations and builds
plugins/	Installed plugins
workspace/	Build workspaces
secrets/	Encrypted credentials and keys
logs/	Log files

Understanding the Jenkins home directory is useful later when diagnosing problems or backing up the system.

1.14 Practical Example of Jenkins

We will use Jenkins as the automation layer that will connect:

- Git for source control,
- Maven for project build and test execution,
- Selenium for browser automation,
- Docker for packaging,
- and a pipeline workflow for orchestrating the whole process.

This means Jenkins is not just a tool for running commands. It is the central engine that coordinates the delivery lifecycle for the project.

Chapter 2 — Jenkins Management and Agent Node Setup

2.1 Why Jenkins Administration Matters?

A Jenkins installation becomes truly useful only when it is managed well. In practice, Jenkins administration includes global system configuration, tool configuration, plugin lifecycle management, credential storage, node management, security settings, and backup-aware operational discipline. Jenkins groups these responsibilities under **Manage Jenkins**, which is the central control area for day-to-day administration.

We are not using Jenkins as a simple build tool. It is the orchestration layer for the full delivery pipeline, so the administrator must understand how Jenkins schedules work, where jobs execute, how agents are added, and how build infrastructure is kept predictable and secure.

2.2 Jenkins Controller, Built-in Node, and Agents?

Jenkins uses a controller/agent architecture. The controller is responsible for coordination, while agents execute builds. Jenkins documentation also refers to the built-in node, which exists inside the controller process itself. Although the built-in node can run tasks, Jenkins discourages using it for builds because of security, performance, and scalability concerns. The documentation explicitly recommends setting the built-in node executors to 0 if you want to prevent task execution there.

NOTE: For sake of simplicity of installation, we may use built-in node for executing the builds. This is also referred as single-node Jenkins. Though Multi-node Jenkins in production is beyond scope of our course, added some part here to have the clarity about the architecture (and the future learning).

This distinction is important for CI/CD design:

- The controller should manage Jenkins.
- Agents should run pipelines, builds, and test workloads.
- Build-heavy jobs should not consume the controller's resources.

This separation keeps the Jenkins UI responsive, reduces operational risk, and makes scaling easier as the number of jobs grows.

2.3 What Jenkins Manages?

The Jenkins administration screen is organised into major control areas. Among the most relevant ones for this syllabus are:

- **System** — global controller settings and paths
- **Tools** — tool locations and automatic installers
- **Plugins** — add, update, remove, enable, or disable plugins
- **Nodes and Clouds** — add, remove, control, and monitor build nodes
- **Security** — controller security settings
- **Manage Credentials** — secure storage and use of credentials

These sections are not just administrative menus. They define the operating model of Jenkins. In a serious CI/CD environment, the controller configuration, tool configuration, and credential configuration must be treated as part of the delivery platform itself.

2.4 Nodes, Executors, and Workload Distribution

A Jenkins **node** is a machine on which build agents run. Jenkins monitors each attached node for disk space, temporary space, swap, clock synchronisation, and response time. If one of these values crosses the configured threshold, Jenkins can mark the node offline.

The number of **executors** configured on a node controls how many tasks that node can run concurrently. In simple terms:

- 1 executor means 1 concurrent build
- 2 executors mean 2 concurrent builds

- 0 executors mean no build execution on that node

This becomes important when designing environments for pipelines that may run Maven builds, Selenium test suites, or Docker image creation in parallel.

A well-designed Jenkins environment usually follows this principle:

- keep the controller lightweight,
 - place workload on agents,
 - assign jobs to agents using labels,
 - and scale agents according to build demand.
-

2.5 Managing Plugins

Plugins are the primary way Jenkins gains functionality. Jenkins documentation describes plugins as the main mechanism for extending Jenkins to suit organisation-specific needs, and it notes that Jenkins has more than a thousand plugins available through the Jenkins ecosystem. Plugins can be installed through the web UI or through the Jenkins CLI, and they are downloaded from the Update Center.

Plugin management is part of normal Jenkins administration:

- install plugins when new capabilities are needed,
- update plugins to pick up fixes and compatibility improvements,
- remove unused plugins to reduce clutter and potential conflicts,
- avoid uncontrolled plugin sprawl.

For use, the most relevant plugin families can be:

- Git and source control plugins,
 - Pipeline plugins,
 - credentials-related plugins,
 - Maven-related plugins,
 - test/reporting plugins used for Selenium integration.
-

2.6 Java Requirement for Jenkins

The Java version requirement is not optional. Jenkins documentation clearly states that unsupported Java versions will prevent the Jenkins controller from running, and the support policy applies to the controller as well as all components of the Jenkins system, including agents and CLI clients. For the current LTS line documented by Jenkins, Java 21 or Java 25 is required.

For us, this matters because the controller and agents should remain compatible with the Jenkins LTS line being used. It also means that when you prepare an agent machine, Java compatibility is part of the setup, not an afterthought.

2.7 Adding a Slave/Agent Node to Jenkins

Earlier Jenkins versions uses the term *slave node*, but Jenkins now uses the term **agent**. The workflow for adding an agent is straightforward and begins from the controller UI.

The flow is:

1. Open **Manage Jenkins**
2. Select **Nodes**
3. Click **New Node**
4. Enter the node name
5. Choose **Permanent Agent**
6. Fill in the agent configuration fields
7. Save the node
8. Launch or reconnect the agent until it becomes online

The key fields on the agent configuration page are:

- **Remote root directory**
- **Labels**
- **Usage**
- **Launch method**
- **Host**
- **Credentials**
- **Host Key verification Strategy**

This is the foundation of node-based pipeline execution. Once the agent is online, Jenkins can delegate jobs to it instead of using the controller.

2.8 Practical Agent Design

The agent can be treated as a dedicated build machine. Considering an Ubuntu-based lab environment, the agent should be prepared with the tools required for the build workload it will handle, such as Java, Maven, Git, Docker, or browser automation dependencies depending on the job stage. The important architectural principle is that the agent must be ready to execute the pipeline cleanly and repeatably.

A good agent design typically follows these rules:

- one agent for general build and test work,
- a separate label for Selenium/browser-related execution if needed
- a defined remote workspace path
- a clear credential-based launch method.

In practice, labels are very important. Labels let you route jobs to the correct machine. For example, a build job can be directed to an agent labelled **linux**, **maven**, or **selenium** depending on what that job requires. Jenkins uses labels to decide where a job may run.

2.9 First Agent Registration Workflow

When a new agent is saved in Jenkins, it is usually registered first and then brought online. Note that, after saving the node, it may appear offline temporarily, and the status page will show the launch process. Once the agent connects successfully, Jenkins reports that it is online.

This behaviour is important because it reflects the actual lifecycle of an agent:

- create node definition,
 - launch agent process,
 - connect agent to controller,
 - verify online status,
 - then assign jobs.
-

2.10 Running Jobs on a Specific Agent

Once an agent is available, jobs can be directed to it using labels. Jenkins documentation shows that a job can be restricted to a node by enabling the option to limit where the project may run and then entering the node label in the label expression field. This ensures that the job executes on the intended machine rather than on the controller or on another available agent.

This capability becomes critical later in the course when we separate responsibilities across the pipeline:

- build jobs can run on one agent,
 - Selenium tests can run on another agent with browser support,
 - deployment jobs can run on a tool-enabled node,
 - the controller remains reserved for orchestration.
-

2.11 Configuration as Code as an Administration Option

Jenkins also supports Configuration as Code through a plugin, allowing controller configuration to be defined in YAML instead of only through the UI. Jenkins documentation describes this as an optional feature for managing controller settings in a human-readable file. This is useful when a team wants repeatable infrastructure and less manual UI configuration drift.

For beginners, it is enough to understand the concept at this stage. The main objective is to recognise that Jenkins can be managed manually through the web UI or declaratively through code-driven configuration.

2.12 Recommended Administration Practices

- Do not overload the controller. Jenkins recommends avoiding build work on the built-in node because it affects performance, security, and scalability.
- Manage plugins carefully. Plugins should be installed only when needed, updated regularly, and removed when unused. Jenkins plugin management is a normal operational task, not a one-time setup activity.
- Manage credentials centrally. Jenkins provides a dedicated credential management area for secure access to third-party systems and applications. This is essential when working with GitHub tokens, Docker registries, deployment targets, etc.

- Use labels and dedicated agents deliberately. A small number of well-configured agents is better than a loosely managed cluster of nodes with unclear roles.

Chapter 3 — Building the First Jenkins Pipeline and Integrating Maven-Based Selenium Tests

This chapter moves from Jenkins administration to actual pipeline implementation. The goal is to understand how Jenkins models a delivery workflow as code, how a **Jenkinsfile** defines the pipeline, how Maven projects are structured for test automation, and how Selenium-based test suites are executed from Jenkins. Jenkins Pipeline is the recommended mechanism for implementing delivery workflows “as code”, and the **Jenkinsfile** is the text file checked into source control that defines the pipeline. Jenkins supports both Declarative and Scripted Pipeline syntax, with Declarative being easier to read and maintain for most delivery workflows.

3.2 From Job Configuration to Pipeline-as-Code

A Jenkins job is the execution unit of automation. In older setups, jobs were often configured directly through the Jenkins UI, but modern Jenkins practice prefers pipeline definitions stored inside the repository. This makes the build logic versioned, reviewable, and consistent with the application source code. Jenkins’ own documentation describes Pipeline as a suite of plugins used to implement and integrate continuous delivery workflows into Jenkins, and it emphasizes that pipelines are written as code using the Pipeline DSL.

For academic and industrial use, the key advantage of pipeline-as-code is traceability. The application source and the delivery process evolve together, so any change to build, test, or deploy logic is committed like regular code. This is especially useful when a project later grows to include separate build, Selenium test, and deployment stages.

3.3 Jenkinsfile as the Heart of the Pipeline

A **Jenkinsfile** is the text file that defines a Jenkins Pipeline and is checked into source control. Jenkins documentation treats it as the standard way to store pipeline logic alongside the application. A Jenkinsfile can be created directly in the repository, or a pipeline can be configured in Jenkins to read it from source control.

Declarative Pipeline is preferred because it provides a structured format with a top-level **pipeline** block, **agent**, **stages**, and **steps**. Jenkins documentation notes that Declarative syntax is designed to make Pipeline code easier to write and read than Scripted syntax. The **step** is the fundamental building block of Pipeline execution, and the **steps** block tells Jenkins what action to perform.

A minimal Declarative Jenkinsfile looks like this:

```
pipeline {
  agent any

  stages {
    stage('Build') {
```

```
    steps {
        sh 'mvn clean compile'
    }
}

stage('Test') {
    steps {
        sh 'mvn test'
    }
}
}
```

This structure will be extended later for packaging, Selenium execution, and deployment-oriented steps. The important idea is that the pipeline is explicit, readable, and repeatable.

3.4 Anatomy of a Delivery Pipeline in Jenkins

A delivery pipeline is the automated path through which source code becomes a validated and deployable software artifact. Jenkins Pipeline is well suited for this model because it allows each stage to be expressed clearly and executed in order. Jenkins' introductory pipeline tutorial demonstrates a basic continuous delivery pipeline and shows how test and delivery stages are added in sequence.

Usually, the logical pipeline flow is:

1. Checkout source code
2. Restore dependencies
3. Build the project
4. Execute tests
5. Collect reports
6. Produce a release artifact
7. Prepare for deployment or downstream integration

The exact implementation will vary by project, but the structural idea remains the same. In a Maven-based Java project, build and test are usually driven through Maven goals, while Selenium tests are typically executed from the test phase using a test runner such as JUnit. Selenium's official documentation describes Selenium as a browser automation project and notes that Java library installation is done through a build tool.

3.5 Creating a Maven Project for Selenium

Maven is the build tool used to manage the Java project structure, dependencies, compilation, and test execution. Apache Maven describes the POM as the fundamental unit of work in Maven and explains that it is the XML file used to describe project configuration. Maven also follows a standard directory layout in which application code typically lives under `src/main/java` and tests live under `src/test/java`.

For Selenium work in Java, Maven is a natural fit because Selenium's Java bindings are installed through a build tool. Selenium's documentation states that installation of Selenium libraries for Java is accomplished

using a build tool, and its WebDriver documentation explains that WebDriver drives a browser natively, either locally or remotely.

A practical Maven project layout for Selenium typically looks like this:

```
selenium-automation-project/  
├── pom.xml  
├── src/  
│   ├── main/  
│   │   └── java/  
│   └── test/  
│       ├── java/  
│       │   └── com/example/tests/  
└── target/
```

In this structure, `pom.xml` defines dependencies and plugins, while `src/test/java` contains Selenium test classes. Maven's standard conventions make the project easier to understand and easier for Jenkins to build consistently.

3.6 Designing the `pom.xml` for Test Automation

The `pom.xml` file is the control center of a Maven project. It describes coordinates, dependencies, plugins, and build behaviour. Maven documentation highlights that the POM contains the information required to build the project and that Maven uses standard conventions for source and test directories.

For a Selenium project, the POM usually includes:

- Selenium Java dependency
- JUnit or TestNG dependency
- Surefire plugin for test execution
- Optional reporting plugins
- Java compiler settings

A typical setup uses the test phase to run browser automation, because Selenium tests belong to the verification layer of the build. Maven's lifecycle documentation explains that Maven's built-in lifecycles include phases that drive project cleaning, testing, and deployment.

A simplified example:

```
<project>  
  <modelVersion>4.0.0</modelVersion>  
  
  <groupId>com.example</groupId>  
  <artifactId>selenium-suite</artifactId>  
  <version>1.0.0</version>  
  
  <properties>  
    <maven.compiler.source>21</maven.compiler.source>
```

```
<maven.compiler.target>21</maven.compiler.target>
</properties>

<dependencies>
  <!-- Selenium and test framework dependencies go here -->
</dependencies>

<build>
  <plugins>
    <!-- Surefire and related plugins go here -->
  </plugins>
</build>
</project>
```

The exact dependency versions can be chosen according to requirements, but the structure above reflects the standard Maven model documented by Apache Maven.

3.7 Adding the Selenium Test Suite

Selenium is used to automate browser actions such as opening pages, clicking elements, and validating expected behaviour. Selenium's official documentation presents WebDriver as the API for driving browsers natively, which makes it suitable for functional and end-to-end testing. Selenium's Java setup is dependency-based, so once the library is added through Maven, test classes can be written under `src/test/java`.

A Selenium test suite should be organised as a proper test layer rather than as ad hoc scripts. This means:

- one test class per test area,
- reusable setup and teardown logic,
- assertions for expected outcomes,
- clear naming for maintainability.

A conceptual example:

```
package com.example.tests;

import org.junit.jupiter.api.Test;

class LoginTest {
    @Test
    void shouldOpenApplicationHomePage() {
        // Selenium steps and assertions
    }
}
```

Later, Jenkins will execute these tests automatically during pipeline runs. The important point is not the specific test case, but the fact that Selenium test code becomes part of the same repository and same build lifecycle as the application code.

3.8 Building the Jenkins Pipeline Job

Jenkins Pipeline jobs are typically created as pipeline projects that read their definition from source control. Jenkins' documentation and tutorials describe creating a Pipeline project and then placing the Jenkinsfile in the repository so the job can run directly from SCM. This is the preferred setup for real projects because the pipeline itself is versioned along with the code.

A pipeline job in Jenkins usually follows this flow:

1. Create a new pipeline job
2. Link the job to the Git repository
3. Configure it to use the Jenkinsfile from SCM
4. Run the first build (manually)
5. Review console output and test reports

The first build verifies that Jenkins can:

- fetch the repository,
- resolve the pipeline definition,
- run the build steps,
- publish the result.

A typical pipeline job configuration in Jenkins is useful when the repository contains:

- application code,
- test code,
- `pom.xml`,
- and `Jenkinsfile`.

That single repository becomes the source of truth for both software and delivery logic.

3.9 Integrating Maven with Jenkins

There are two common ways to run Maven from Jenkins pipelines. The simplest is to invoke Maven directly through shell steps. Jenkins' pipeline documentation and Maven tutorial both show that Jenkins can build Java projects with Maven through Pipeline jobs. Another option is the Pipeline Maven Plugin, which provides the `withMaven` step to configure a Maven environment inside a pipeline and improve traceability of Maven builds.

The `withMaven` step is valuable when you want Jenkins to manage Maven installation and include Maven build metadata in a pipeline-aware way. The plugin documentation states that `withMaven` configures a Maven environment and prepends the selected Maven installation to the path.

A practical pipeline section may look like this:

```
stage('Test') {  
    steps {  
        sh 'mvn test'  
    }  
}
```

Or, when using the Maven pipeline plugin:

```
stage('Test') {  
    steps {  
        withMaven {  
            sh 'mvn test'  
        }  
    }  
}
```

Both approaches are valid in a learning environment. The first keeps things simple, while the second provides richer Maven integration.

3.10 Publishing Test Results

A Jenkins pipeline should not only run tests, but also expose the results clearly. Jenkins supports test result publishing through Pipeline steps and plugins, and the general pipeline ecosystem includes steps designed to integrate with build outputs and reports. The [Jenkins Java Maven tutorial](#) notes that test results can be saved as JUnit XML reports, which Jenkins can then display in the build history.

A typical practice is to collect JUnit-style reports after the test stage:

```
post {  
    always {  
        junit 'target/surefire-reports/*.xml'  
    }  
}
```

This gives developers fast visibility into:

- passed tests,
- failed tests,
- flaky tests,
- and historical test trends.

3.11 Sample End-to-End Jenkinsfile

The following Jenkinsfile represents a first realistic pipeline for a Maven-based Selenium project:

```
pipeline {
  agent any

  tools {
    jdk 'JDK21'
    maven 'Maven3'
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }

    stage('Build') {
      steps {
        sh 'mvn clean compile'
      }
    }

    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
  }

  post {
    always {
      junit 'target/surefire-reports/*.xml'
    }
  }
}
```

This example uses the standard Declarative Pipeline structure described in Jenkins documentation. It shows the main ideas that matter at this stage of the course: source checkout, Maven build, automated test execution, and test report publication.

3.12 What to Verify After the First Run

After the first build runs successfully, the Jenkins console should confirm that:

- the repository was checked out,
- Maven executed correctly,
- the test phase completed,
- and the test reports were collected.

If the build fails, the usual causes are:

- incorrect repository URL,
- missing credentials,
- Maven not available on the node,
- Java version mismatch,
- test classpath issues,
- or Selenium dependency errors. These are normal integration problems and part of the learning process for CI/CD practice. Jenkins documents its pipeline development tools and Pipeline steps reference to help diagnose and refine such jobs.

Chapter 4 — Selenium Integration with Jenkins and the Complete Pipeline Flow

This chapter completes the a pipeline workflow by connecting Jenkins, Maven, and Selenium into one automated workflow. The central idea is simple: a Jenkins pipeline should be able to fetch the code from version control, build the Maven project, execute the Selenium test suite, publish the results, and keep the delivery process repeatable. Jenkins Pipeline is designed exactly for this kind of workflow, with the `Jenkinsfile` acting as the source-controlled definition of the build, test, and delivery process.

4.2 Why Selenium Belongs in the Delivery Pipeline?

Selenium is used for browser automation and is primarily intended for testing web applications. Selenium WebDriver drives browsers natively, either locally or on a remote machine using the Selenium server. That makes it suitable for validating real user flows such as login, form submission, navigation, and page-level behaviour.

In a CI/CD pipeline, Selenium tests act as a functional verification layer. Unit tests confirm that code behaves correctly in isolation, while Selenium confirms that the application works correctly through the browser interface. In practice, this is the stage that gives the team confidence that the application is not only compiling, but also usable from the end-user perspective.

4.3 Preparing the Selenium Project for Jenkins

A Selenium project in Java should be organised using Maven conventions. Maven's standard layout places application sources under `src/main/java`, test sources under `src/test/java`, and build output under `target`. The `pom.xml` file is the project's declarative configuration and is the fundamental unit of work in Maven.

For this course, the most natural structure is:

```
project-root/
├── pom.xml
├── src/
│   ├── main/
│   │   └── java/
```

```
├── test/
│   ├── java/
│   └── tests/
└── target/
```

This layout is important because Jenkins will later execute Maven goals such as `test` or `clean test`, and Maven will look in the expected source and test directories automatically.

4.4 Configuring the Maven Build for Selenium Tests

The Maven POM should declare the test framework, Selenium dependency, and the plugin that runs test cases during the build lifecycle. Maven's documentation describes the POM as the place where project configuration and plugin settings are declared, and the build lifecycle is driven from that configuration.

A typical build execution pattern is:

```
mvn clean test
```

This command removes old build output and then runs the test phase. Jenkins can call this command directly in a pipeline stage, making the Maven build part of the automated delivery path.

4.5 Writing the Selenium Test Suite

Selenium WebDriver is the browser-driving API used for writing test cases. The WebDriver documentation describes it as a browser automation API that drives browsers natively, either locally or through Selenium Server on a remote machine. This makes it suitable for UI validation in a pipeline environment.

A good Selenium suite should be: modular, readable, deterministic, and easy to run.

A simple Java test class may look like this:

```
package com.example.tests;

import org.junit.jupiter.api.Test;

class LoginTest {

    @Test
    void shouldLoadHomePage() {
        // Open browser
        // Navigate to application URL
        // Validate title or key UI element
    }
}
```

From Jenkins' perspective, these are simply Maven test cases. From the application team's perspective, these are end-to-end functional checks that guard the user interface.

4.6 The Role of Jenkins Agents in Selenium Execution

A Selenium test stage often needs a node that has the right runtime tools available. Jenkins labels are used to group agents based on criteria such as operating system or hardware capability, and they help assign jobs to the correct node. Jenkins also documents that executors on a node determine how many tasks can run concurrently.

For Selenium work, a practical approach is to use a dedicated agent label such as `selenium` or `browser`, then direct the pipeline stage to that label. This keeps UI testing isolated from unrelated build workloads and ensures the stage runs on a machine prepared for browser execution. Jenkins Pipeline also supports node allocation so that a stage runs inside a workspace on a specific agent.

4.7 Integrating Selenium into the Jenkinsfile

Declarative Pipeline requires the entire pipeline to be wrapped in a `pipeline` block. Jenkins documentation states that Declarative syntax is a more simplified and opinionated form of Pipeline, and stages are defined with `stage` blocks. The `sh` step executes shell commands, and the `junit` step records test reports for Jenkins to display.

A practical Jenkinsfile for the Selenium project can be structured like this:

```
pipeline {
  agent none

  stages {
    stage('Checkout') {
      agent any
      steps {
        checkout scm
      }
    }

    stage('Build') {
      agent any
      steps {
        sh 'mvn clean compile'
      }
    }

    stage('Selenium Tests') {
      agent { label 'selenium' }
      steps {
        sh 'mvn test'
      }
    }
  }
}
```

```
    post {
        always {
            junit 'target/surefire-reports/*.xml'
        }
    }
}
```

This structure separates the pipeline into logical responsibilities. The code checkout and compile steps can run on a general-purpose agent, while the Selenium stage can be pinned to a node with browser support. The `post` section then collects JUnit XML reports so Jenkins can display test results and trends.

4.8 Recording and Visualising Test Results

A Jenkins pipeline should not stop at execution. It must also record test outcomes. Jenkins' JUnit step is designed to collect XML test reports, and Jenkins documentation explains that when tests fail, the Pipeline can be marked unstable while still preserving report data in the UI. This is useful because failing test suites are part of the normal feedback loop in CI/CD.

The recommended pattern is to publish test results in the `post { always { ... } }` block, so they are archived whether the stage passes or fails. Jenkins also provides historical reporting and trend analysis once the results are stored.

This matters for Selenium because browser tests are often the first automated signal that a recent code change has broken a user flow. Jenkins gives the team visibility into that failure instead of hiding it inside the console output.

4.9 Using Credentials in the Pipeline

Real pipelines often need secure access to external systems such as Git repositories, artifact registries, cloud storage, or databases. Jenkins documentation recommends using credentials for this purpose instead of hardcoding secrets into the pipeline or source code. Jenkins also provides the `withCredentials` step for binding secrets to environment variables inside a controlled scope.

For this course, credentials become relevant when the pipeline later needs to:

- authenticate to GitHub,
- push or pull artifacts,
- connect to private services,
- or deploy to infrastructure.

Even when Selenium tests themselves do not require secrets, the surrounding pipeline often does. That is why Jenkins credentials management is part of the delivery pipeline story rather than a separate administrative topic.

4.10 A Practical Execution Flow

A complete Jenkins-based execution flow for the Selenium-enabled Maven project follows this sequence:

1. Developer commits code to Git.
2. Jenkins detects the change and starts the pipeline.
3. Jenkins checks out the repository.
4. Maven compiles the application.
5. Maven runs the Selenium test suite.
6. Jenkins records test reports.
7. The pipeline ends with a visible build result.

This workflow reflects the central purpose of Jenkins as an automation server for building, testing, and delivering software. It also shows how Selenium fits into CI/CD as a browser-level verification layer rather than an isolated manual testing activity.
